

DESIGN AND VERIFICATION OF A 32-BIT SINGLE-CYCLE RISC-V PROCESSOR USING VERILOG HDL

A Processor designed in Verilog with ALU, register file, memory interfaces, branch logic, and SystemVerilog.

Project Title

A 32-Bit Single-Cycle Risc-V Processor

Engineer

Kishore Vanapalli

Domain

VLSI Design and Verification

Languages Used

Verilog HDL, SystemVerilog

Tools Used

ModelSim/QuestaSim, GitHub

Focus Areas

RTL Design, Clock Domain Crossing (CDC), Functional Verification,
Digital Design

Table of Contents

Primary Page	1
Table of Contents	2
Abstract	7
Chapter 1: Introduction	8
1.1 Introduction to Processors	8
1.2 Evolution of RISC Architecture	8
1.3 Introduction to RISC-V	9
1.4 Need for Processor Design	9
1.5 Applications of RISC-V Processors	10
Chapter 2: Problem Statement	12
Chapter 3: Objectives	13
Design Objectives	13
Functional Objectives	13
Verification Objectives	13
Chapter 4: Introduction to RISC-V ISA	14
4.1 What is an Instruction Set Architecture?	14
4.2 Overview of RISC-V ISA	14
4.3 Advantages of RISC-V	15
4.4 RV32I Base Integer ISA	15
4.5 Register Organization	17
Chapter 5: Processor Specifications	18
Chapter 6: Supported Instructions	19
Arithmetic Instructions	19
Immediate Instructions	19
Memory Instructions	19
Control Instructions	19
Chapter 7: Processor Architecture	20
Top-Level Architecture	20
Chapter 8: Datapath Design	22
Stage 1: Instruction Fetch	22
Stage 2: Instruction Decode	22
Stage 3: Execute	22
Stage 4: Memory Access	22

Stage 5: Write Back.....	22
Datapath Flow	22
Chapter 9: Program Counter (PC)	24
9.1 Introduction.....	24
9.2 Functions of Program Counter.....	24
9.3 Sequential Execution.....	24
9.4 Branch Execution	24
9.5 Jump Execution	25
Program Counter Flow.....	25
Chapter 10: Instruction Memory	26
10.1 Introduction.....	26
10.2 Need for Instruction Memory	26
10.3 Architecture.....	26
10.4 Instruction Fetch Operation.....	26
Instruction Memory Diagram	27
10.5 Advantages.....	27
Chapter 11: Register File	28
11.1 Introduction.....	28
11.2 Register Organization	28
11.3 Register Ports	28
Register File Diagram.....	29
11.4 Register Operations	29
11.5 Importance of Register File	29
Chapter 12: Arithmetic Logic Unit (ALU)	30
12.1 Introduction.....	30
12.2 Inputs and Outputs	30
12.3 Supported Operations	30
ALU Control Encoding.....	31
12.4 Zero Flag	31
ALU Architecture	31
Chapter 13: Control Unit	33
13.1 Introduction.....	33
13.2 Functions	33
13.3 Inputs	33

13.4 Outputs	33
Control Unit Flow	34
Chapter 14: Immediate Generator	35
14.1 Introduction.....	35
14.2 Need for Immediate Generator	35
14.3 I-Type Immediate.....	35
14.4 S-Type Immediate.....	36
14.5 B-Type Immediate	36
14.6 J-Type Immediate	36
Immediate Generator Flow	37
Chapter 15: Data Memory	38
15.1 Introduction.....	38
15.2 Memory Operations.....	38
15.3 Load Word (LW)	38
15.4 Store Word (SW)	38
Data Memory Diagram	39
15.5 Advantages.....	39
Chapter 16: Branch Unit	40
16.1 Introduction.....	40
16.2 Need for Branch Unit	40
16.3 BEQ Instruction.....	40
16.4 BNE Instruction.....	40
16.5 JAL Instruction.....	41
16.6 Next PC Logic	41
Branch Unit Flow.....	41
Chapter 17: Verification Methodology	42
17.1 Introduction.....	42
17.2 Verification Flow	42
17.3 Testbench Architecture	42
17.4 Verification Objectives	43
17.6 Verification Strategy	43
17.5 Signals Monitored	44
Chapter 18: Test Case 1 – Arithmetic Operations	45
18.1 Objective.....	45

18.2 ADD Instruction Verification	45
18.3 SUB Instruction Verification	45
18.4 AND Instruction Verification	46
18.5 OR Instruction Verification	46
18.6 XOR Instruction Verification	47
Arithmetic Verification Summary	47
Chapter 19: Test Case 2 – Immediate Instructions	48
19.1 Objective.....	48
19.2 ADDI Verification	48
19.3 ORI Verification	48
19.4 ANDI Verification	49
Immediate Verification Summary	49
Chapter 20: Test Case 3 – Memory Operations	50
20.1 Objective.....	50
20.2 Store Word Verification.....	50
20.3 Load Word Verification	50
Memory Verification Summary	51
Chapter 21: Test Case 4 – Branch and Jump Operations	52
21.1 Objective.....	52
21.2 BEQ Verification	52
21.3 BNE Verification	52
21.4 JAL Verification	53
Branch Verification Summary	53
Chapter 22: Complete Program Execution	54
22.1 Objective.....	54
Program 1: Sum of Array	54
Program 2: Factorial Program.....	54
Program 3: Counter Program	54
Program Execution Summary	55
Chapter 23: Simulation Results	56
23.1 Simulation Environment	56
23.2 Waveform Analysis.....	56
23.3 Terminal Output	58
23.4 Functional Coverage Summary.....	58

Chapter 24: Applications	59
24.1 Introduction.....	59
24.2 Embedded Systems	59
24.3 Internet of Things (IoT)	59
24.4 FPGA-Based Systems.....	60
24.5 Automotive Electronics	61
24.6 Industrial Automation.....	61
24.7 Educational and Research Applications	62
Application Summary	62
Chapter 25: Advantages and Limitations	63
25.1 Advantages.....	63
25.2 Limitations	64
Advantages and Limitations Summary	65
Chapter 26: Future Scope.....	66
26.1 Complete RV32I Support.....	66
26.2 Multiplication and Division Instructions	66
26.3 Pipelined Architecture	66
26.4 Cache Memory	67
26.5 Interrupt and Exception Handling	67
26.6 Branch Prediction.....	67
26.7 FPGA Implementation.....	68
26.8 UVM-Based Verification	68
26.9 Multicore RISC-V Systems	68
Chapter 27: Conclusion.....	70
Chapter 28: Interview Questions and Viva Preparation	72
Basic Questions	72
Architecture Questions.....	73
Instruction Questions	74
Verification Questions	76
Expected Interview Outcome	77

Abstract

The RISC-V instruction set architecture has gained significant attention in academia and industry due to its open-source nature, simplicity, and scalability. This project presents the design and verification of a **32-bit Single-Cycle RISC-V Processor** based on the **RV32I instruction set** using **Verilog HDL** and **SystemVerilog**. The processor executes each instruction in a single clock cycle and incorporates essential CPU components, including the Program Counter (PC), Instruction Memory, Register File, Arithmetic Logic Unit (ALU), Control Unit, Immediate Generator, Data Memory, and Branch Unit.

The implemented processor supports fundamental arithmetic, logical, memory access, and branch instructions such as **ADD, SUB, AND, OR, XOR, ADDI, LW, SW, and BEQ**. A comprehensive verification environment was developed using SystemVerilog testbenches to validate functional correctness under different execution scenarios. Simulation and waveform analysis were performed using **ModelSim Intel FPGA Edition**, confirming correct instruction execution, register updates, memory operations, and branch decisions.

This project demonstrates the practical implementation of processor architecture concepts and provides a strong foundation for advanced CPU designs, including pipelined and multicore RISC-V processors. The design serves as an educational and research-oriented platform for learning **RTL design, processor architecture, digital system design, and verification methodologies**.

Chapter 1: Introduction

1.1 Introduction to Processors

A processor, also known as the Central Processing Unit (CPU), is the computational core of a digital system responsible for executing instructions and processing data. It performs arithmetic operations, logical operations, memory accesses, and control decisions required to execute software programs.

Modern processors are used in a wide range of applications, including personal computers, smartphones, embedded systems, automotive electronics, industrial automation systems, and Internet of Things (IoT) devices.

A processor generally performs the following operations:

- Fetches instructions from memory.
- Decodes instructions.
- Executes arithmetic and logical operations.
- Reads and writes data from memory.
- Updates the Program Counter (PC).
- Controls the overall execution flow.

1.2 Evolution of RISC Architecture

Early processors were based on Complex Instruction Set Computing (CISC) architectures, where instructions could perform multiple operations with varying instruction lengths. Although powerful, these processors required complex hardware implementation.

To overcome these challenges, Reduced Instruction Set Computing (RISC) architectures were introduced.

RISC architectures are based on the following principles:

- Simple instruction formats
- Fixed instruction lengths

- Load-store architecture
- Large register files
- Simplified control logic
- Faster instruction execution

1.3 Introduction to RISC-V

RISC-V is an open-standard Instruction Set Architecture (ISA) based on RISC principles. It was developed at the University of California, Berkeley, to provide a free and extensible processor architecture.

Unlike proprietary architectures, RISC-V allows engineers and organizations to design processors without licensing restrictions.

RISC-V provides:

- Open-source ISA
- Modular architecture
- Fixed-length instructions
- Simplified decoding
- Extensible instruction sets
- Easy customization

The architecture is organized into several standard instruction sets:

- RV32I – 32-bit Integer Base ISA
- RV64I – 64-bit Integer Base ISA
- RV128I – 128-bit Integer Base ISA

This project implements a subset of the RV32I instruction set.

1.4 Need for Processor Design

Modern digital systems require processors that are:

- High-performance
- Low-power
- Scalable
- Modular
- Easy to verify
- Customizable

Understanding processor design enables engineers to:

- Develop custom processors
- Build application-specific computing systems
- Understand computer architecture concepts
- Design advanced processors such as pipelined CPUs and System-on-Chip (SoC) architectures

Processor design forms the foundation of:

- FPGA development
- ASIC design
- Embedded systems engineering
- Computer architecture research
- Hardware verification engineering

1.5 Applications of RISC-V Processors

RISC-V processors are widely used in:

Embedded Systems

Microcontrollers and embedded devices require lightweight and customizable processors.

Internet of Things (IoT)

RISC-V processors provide low-power computing solutions for IoT devices.

Automotive Electronics

Modern vehicles use processors for engine control, safety systems, and autonomous driving applications.

FPGA-Based Systems

RISC-V processors are commonly implemented on FPGA platforms for research and prototyping.

Industrial Automation

Industrial control systems use processors for monitoring and control applications.

Research and Education

Because of its open nature, RISC-V is extensively used in universities and research laboratories.

Chapter 2: Problem Statement

Modern computing systems require processors that can efficiently execute instructions while maintaining simplicity and flexibility.

Traditional proprietary processor architectures often involve:

- High licensing costs
- Limited customization
- Complex hardware implementation
- Restricted architectural modifications

Educational institutions and research organizations require processors that:

- Are easy to understand
- Support modular design methodologies
- Allow architectural modifications
- Can be implemented on FPGA platforms
- Demonstrate fundamental processor concepts

Therefore, there is a need to design a processor that:

- Implements a standard instruction set
- Supports arithmetic and memory operations
- Supports control instructions
- Demonstrates processor datapath and control logic
- Is fully synthesizable and reusable

This project addresses these requirements by implementing a 32-bit Single-Cycle RISC-V Processor using Verilog HDL.

Chapter 3: Objectives

The primary objective of this project is to design and verify a 32-bit Single-Cycle RISC-V Processor implementing a subset of the RV32I instruction set.

Design Objectives

1. Design a 32-bit processor architecture.
2. Implement modular RTL components.
3. Develop processor datapath and control logic.
4. Support arithmetic, memory, and branch instructions.

Functional Objectives

1. Implement Program Counter operation.
2. Implement instruction fetching.
3. Implement register file operations.
4. Implement ALU operations.
5. Implement memory read and write operations.
6. Implement branch and jump instructions.

Verification Objectives

1. Develop SystemVerilog testbench.
2. Verify arithmetic instructions.
3. Verify memory instructions.
4. Verify branch instructions.
5. Verify complete program execution.
6. Analyze simulation waveforms.

Chapter 4: Introduction to RISC-V ISA

4.1 What is an Instruction Set Architecture?

An Instruction Set Architecture (ISA) defines the interface between software and hardware.

The ISA specifies:

- Supported instructions
- Register organization
- Instruction formats
- Memory addressing methods
- Exception handling mechanisms

The ISA acts as a contract between the processor hardware and software applications.

4.2 Overview of RISC-V ISA

RISC-V is based on the following design principles:

- Simplicity
- Modularity
- Extensibility
- Efficient hardware implementation
- Open-source availability

The instruction execution flow is:

Application Program



Compiler



RISC-V Instructions



Processor Hardware

4.3 Advantages of RISC-V

Open Standard

No licensing requirements.

Modular Design

Additional instruction extensions can be added.

Simple Instruction Encoding

Simplifies hardware implementation.

Fixed-Length Instructions

Simplifies instruction fetching and decoding.

Extensibility

Allows application-specific processor customization.

Educational Value

Provides excellent learning opportunities for processor design and verification.

4.4 RV32I Base Integer ISA

RV32I represents:

RV = RISC-V

32 = 32-bit architecture

I = Integer Base Instruction Set

RV32I contains:

- Integer arithmetic instructions
- Logical instructions

- Immediate instructions
- Memory instructions
- Branch instructions
- Jump instructions

This project implements the following instructions:

Arithmetic Instructions

ADD

SUB

AND

OR

XOR

Immediate Instructions

ADDI

ORI

ANDI

Memory Instructions

LW

SW

Control Instructions

BEQ

BNE

JAL

4.5 Register Organization

RV32I contains thirty-two general-purpose registers:

x0
x1
x2
...
x31

Each register is:

- 32 bits wide
- Used for data storage
- Accessible by instructions

Special Register:

x0 = Constant Zero

The large register set reduces memory accesses and improves processor efficiency.

Chapter 5: Processor Specifications

The processor developed in this project has the following specifications.

Parameter	Specification
Processor Architecture	Single-Cycle RISC-V
ISA	RV32I Subset
Data Width	32 Bits
Number of Registers	32
Register Width	32 Bits
Instruction Width	32 Bits
Program Counter Width	32 Bits
Instruction Memory	Separate Memory
Data Memory	Separate Memory
RTL Language	Verilog HDL
Verification Language	SystemVerilog
Simulator	ModelSim/QuestaSim
Design Methodology	Modular RTL Design

Chapter 6: Supported Instructions

The processor supports a subset of RV32I instructions.

Arithmetic Instructions

Instruction	Description
ADD	Addition
SUB	Subtraction
AND	Logical AND
OR	Logical OR
XOR	Logical XOR

Immediate Instructions

Instruction	Description
ADDI	Add Immediate
ORI	OR Immediate
ANDI	AND Immediate

Memory Instructions

Instruction	Description
LW	Load Word
SW	Store Word

Control Instructions

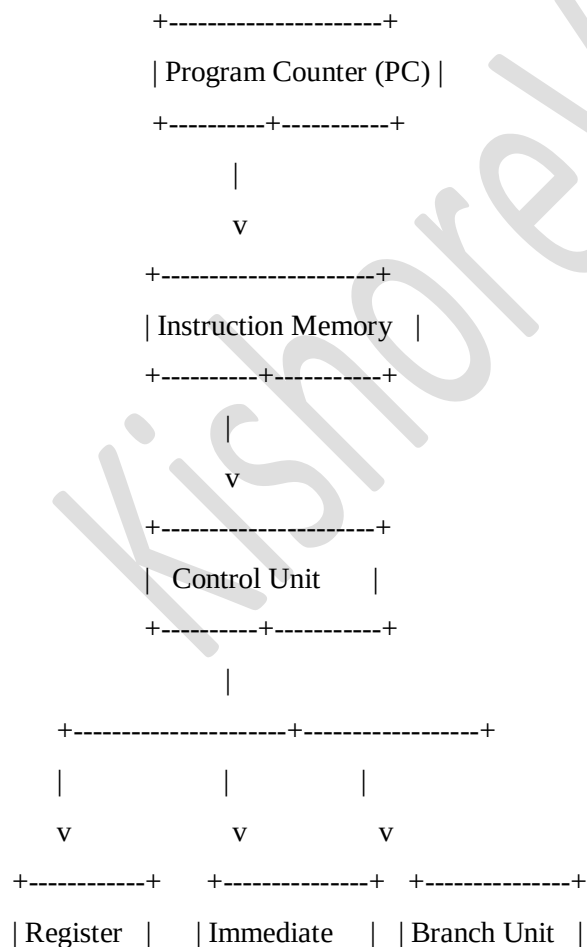
Instruction	Description
BEQ	Branch if Equal
BNE	Branch if Not Equal
JAL	Jump and Link

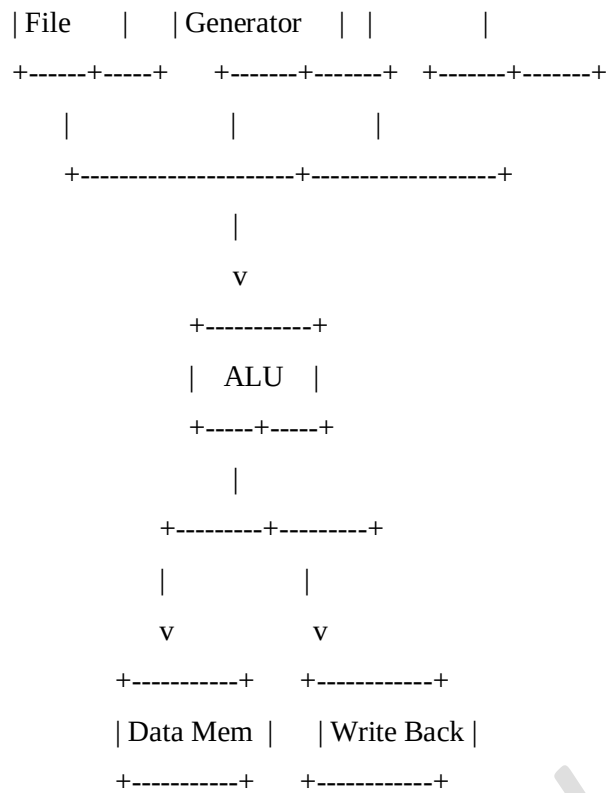
Chapter 7: Processor Architecture

The processor consists of the following major modules:

1. Program Counter
2. Instruction Memory
3. Register File
4. Control Unit
5. Immediate Generator
6. Arithmetic Logic Unit
7. Data Memory
8. Branch Unit

Top-Level Architecture





The processor fetches instructions, decodes them, executes operations, accesses memory if required, and writes results back to the register file in a single clock cycle.

Chapter 8: Datapath Design

The datapath is the hardware path through which instructions and data flow during processor execution.

The processor executes instructions through five logical stages.

Stage 1: Instruction Fetch

The Program Counter provides the address of the current instruction.

Stage 2: Instruction Decode

The instruction fields are decoded and operands are read from the register file.

Stage 3: Execute

The ALU performs arithmetic and logical operations.

Stage 4: Memory Access

Load and Store instructions access data memory.

Stage 5: Write Back

Results are written back to the destination register.

Datapath Flow

Program Counter



Instruction Memory



Instruction Decode



Register File



Arithmetic Logic Unit



Data Memory



Write Back

The single-cycle architecture completes all these operations within one clock period, making the design simple and easy to understand while serving as a strong foundation for advanced processor architectures such as pipelined and superscalar processors.

Chapter 9: Program Counter (PC)

9.1 Introduction

The Program Counter (PC) is one of the most important registers in a processor. It stores the address of the instruction currently being executed and controls the sequential flow of program execution.

Every instruction execution begins by fetching an instruction from the address stored in the Program Counter.

9.2 Functions of Program Counter

The Program Counter performs the following functions:

- Stores the current instruction address
- Generates the next instruction address
- Supports sequential execution
- Supports branch execution
- Supports jump execution

9.3 Sequential Execution

For normal instructions, the processor executes instructions sequentially.

The next instruction address is:

$$PC_{next} = PC + 4$$

Since every RISC-V instruction is 32 bits (4 bytes), the Program Counter increments by four.

9.4 Branch Execution

During branch instructions:

BEQ

BNE

The Program Counter may change to:

$$PC_{next} = PC + \text{Branch Offset}$$

This enables conditional execution of instructions.

9.5 Jump Execution

For JAL instruction:

$$PC_{next} = PC + \text{Jump Offset}$$

The return address:

$$PC + 4$$

is stored in the destination register.

Program Counter Flow

Current PC

|

v

Instruction Fetch

|

v

Branch/Jump Decision

|

v

Next PC Selection

|

v

Updated PC

Chapter 10: Instruction Memory

10.1 Introduction

Instruction Memory stores the program instructions that are executed by the processor.

The Program Counter provides the memory address and the corresponding instruction is fetched from memory.

10.2 Need for Instruction Memory

Without instruction memory:

- Processor cannot fetch instructions
- Program execution is impossible
- Datapath becomes non-functional

Instruction memory acts as the software storage unit of the processor.

10.3 Architecture

Inputs:

- Address (Program Counter)

Output:

- 32-bit Instruction

10.4 Instruction Fetch Operation

Step 1:

PC = Current Address

Step 2:

Instruction = IMEM[PC]

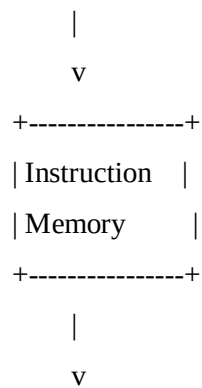
Step 3:

Instruction is sent to:

- Control Unit
- Register File
- Immediate Generator

Instruction Memory Diagram

Program Counter



32-bit Instruction

10.5 Advantages

- Fast instruction fetching
- Simple addressing mechanism
- Easy program loading
- Suitable for simulation and FPGA implementation

Chapter 11: Register File

11.1 Introduction

The Register File is a collection of high-speed storage elements inside the processor.

Registers store:

- Operands
- Intermediate results
- Addresses
- Function parameters

The Register File significantly improves processor performance because accessing registers is much faster than accessing memory.

11.2 Register Organization

RV32I contains:

32 Registers

Each register width:

32 Bits

Registers:

x0 to x31

Special register:

x0 = Constant Zero

11.3 Register Ports

The Register File contains:

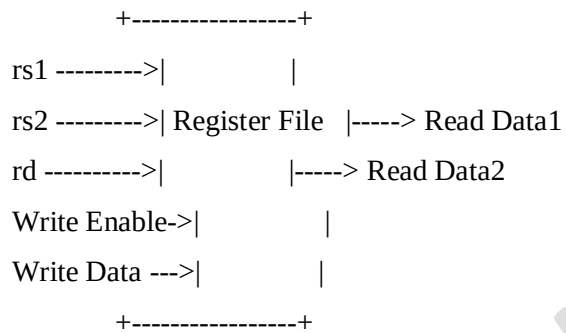
Two Read Ports

- Read Data 1
- Read Data 2

One Write Port

- Write Data

Register File Diagram



11.4 Register Operations

Read Operation:

ReadData1 = Register[rs1]

ReadData2 = Register[rs2]

Write Operation:

Register[rd] = WriteData

11.5 Importance of Register File

- High-speed data storage
- Reduces memory accesses
- Improves processor efficiency
- Supports arithmetic operations

Chapter 12: Arithmetic Logic Unit (ALU)

12.1 Introduction

The Arithmetic Logic Unit (ALU) performs all arithmetic and logical operations in the processor.

It is considered the computational engine of the processor.

12.2 Inputs and Outputs

Inputs:

- Operand A
- Operand B
- ALU Control

Outputs:

- Result
- Zero Flag

12.3 Supported Operations

Addition

Result = $A + B$

Subtraction

Result = $A - B$

AND

Result = $A \& B$

OR

Result = $A | B$

XOR

Result = $A \wedge B$

ALU Control Encoding

Operation	Control
ADD	000
SUB	001
AND	010
OR	011
XOR	100

12.4 Zero Flag

The Zero Flag is generated when:

Result == 0

The Zero Flag is primarily used for:

- BEQ instruction
- BNE instruction

ALU Architecture

Operand A

|

v

+-----+

| ALU |

+-----+

^

|

Operand B

Outputs:

Result

Zero Flag

Kishore/Vanapalli

Chapter 13: Control Unit

13.1 Introduction

The Control Unit is responsible for generating control signals that determine how every module in the processor operates.

It acts as the brain of the processor.

13.2 Functions

The Control Unit:

- Decodes instructions
- Generates control signals
- Controls datapath operation
- Controls memory access
- Controls register writes
- Controls branching

13.3 Inputs

- Opcode
- funct3
- funct7

13.4 Outputs

RegWrite

Enables register writing.

MemRead

Enables memory reading.

MemWrite

Enables memory writing.

MemToReg

Selects memory output.

ALUSrc

Selects ALU operand source.

Branch

Enables branching.

Jump

Enables jump instructions.

ALUControl

Selects ALU operation.

Control Unit Flow

Instruction

|

v

Opcode Decode

|

v

Generate Control Signals

|

v

Control Datapath

Chapter 14: Immediate Generator

14.1 Introduction

Several RISC-V instructions contain immediate values embedded inside the instruction.

The Immediate Generator extracts these values and converts them into 32-bit signed values.

14.2 Need for Immediate Generator

Immediate values are required for:

- ADDI
- ORI
- ANDI
- LW
- SW
- BEQ
- BNE
- JAL

14.3 I-Type Immediate

Used for:

ADDI

ORI

ANDI

LW

Immediate:

Instruction[31:20]

14.4 S-Type Immediate

Used for:

SW

Immediate:

Instruction[31:25]

Instruction[11:7]

14.5 B-Type Immediate

Used for:

BEQ

BNE

Immediate:

Instruction[31]

Instruction[7]

Instruction[30:25]

Instruction[11:8]

14.6 J-Type Immediate

Used for:

JAL

Immediate:

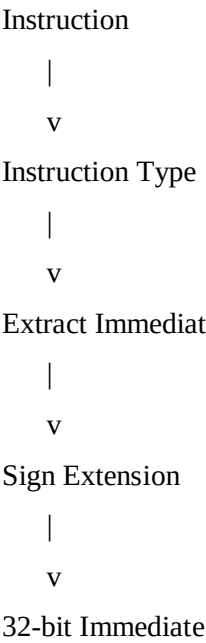
Instruction[31]

Instruction[19:12]

Instruction[20]

Instruction[30:21]

Immediate Generator Flow



Chapter 15: Data Memory

15.1 Introduction

Data Memory stores data that is loaded and stored during program execution.

It acts as the data storage unit of the processor.

15.2 Memory Operations

The processor supports:

LW

SW

15.3 Load Word (LW)

Operation:

Register = Memory[Address]

Steps:

9. Calculate address.
10. Read memory data.
11. Write data into register.

15.4 Store Word (SW)

Operation:

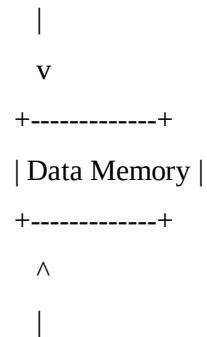
Memory[Address] = Register Data

Steps:

12. Calculate address.
13. Read register data.
14. Store data into memory.

Data Memory Diagram

Address



Data In

Output:

Data Out

15.5 Advantages

- Supports program data storage
- Enables array processing
- Enables variable storage
- Supports software execution

Chapter 16: Branch Unit

16.1 Introduction

The Branch Unit determines the next instruction address based on branch and jump instructions.

It controls the execution flow of the processor.

16.2 Need for Branch Unit

Without branching:

- Loops cannot be implemented
- Conditional statements cannot execute
- Functions cannot be called

Branching is essential for program execution.

16.3 BEQ Instruction

Operation:

if(rs1 == rs2)

 PC = Target Address

else

 PC = PC + 4

16.4 BNE Instruction

Operation:

if(rs1 != rs2)

 PC = Target Address

else

 PC = PC + 4

16.5 JAL Instruction

Operation:

PC = Target Address

rd = PC + 4

16.6 Next PC Logic

Sequential:

PCnext = PC + 4

Branch:

PCnext = PC + Offset

Jump:

PCnext = PC + Offset

Branch Unit Flow

ALU Result

|

v

Comparison Logic

|

v

Branch Decision

|

v

Next PC Selection

|

v

Updated Program Counter

Chapter 17: Verification Methodology

17.1 Introduction

Verification is the process of ensuring that the designed hardware behaves according to its specifications. In processor design, verification is extremely important because a small design error can lead to incorrect program execution.

The objective of verification is to ensure that every instruction is executed correctly and that all processor modules interact properly.

This project uses:

- Verilog HDL for RTL Design
- SystemVerilog for Testbench Development
- ModelSim/QuestaSim for Simulation and Debugging

17.2 Verification Flow

The processor verification flow is shown below:

RTL Design > Compilation > Testbench Development > Simulation > Waveform Analysis
> Debugging > PASS/FAIL Verification

17.3 Testbench Architecture

The SystemVerilog testbench performs the following tasks:

Clock Generation

Generates the system clock required for processor operation.

Reset Generation

Initializes all processor modules into known states.

Program Loading

Loads instructions into instruction memory.

Monitoring

Monitors processor outputs such as:

- Program Counter
- Instructions
- Register Values
- ALU Results
- Memory Data
- Control Signals

Self-Checking

Automatically compares expected and actual outputs.

17.4 Verification Objectives

The processor is verified for:

- Arithmetic instructions
- Immediate instructions
- Memory operations
- Branch operations
- Jump operations
- Complete program execution

17.6 Verification Strategy

The verification methodology follows a bottom-up approach:

15. Verify individual modules.

16. Integrate modules.
17. Verify instruction execution.
18. Verify complete programs.
19. Analyze waveforms.
20. Confirm expected results.

17.5 Signals Monitored

Signal	Purpose
clk	Processor clock
reset	Processor reset
pc	Current instruction address
instruction	Current instruction
rs1_data	Operand 1
rs2_data	Operand 2
alu_result	ALU output
mem_data	Data memory output
rd_data	Register write-back data

Chapter 18: Test Case 1 – Arithmetic Operations

18.1 Objective

Verify correct execution of arithmetic and logical instructions.

Supported instructions:

- ADD
- SUB
- AND
- OR
- XOR

18.2 ADD Instruction Verification

Instruction:

text id="j3x6n9" ADD x3, x1, x2

Initial Values:

text id="e6m2q1" x1 = 10 x2 = 20

Expected Result:

text id="v4d9p7" x3 = 30

Verification Status:

PASS

18.3 SUB Instruction Verification

Instruction:

text id="o1k8c5" SUB x3, x1, x2

Initial Values:

text id="x7b3m4" x1 = 50 x2 = 30

Expected Result:

text id="g9s6w2" x3 = 20

Verification Status:

PASS

18.4 AND Instruction Verification

Instruction:

text id="m5h1q8" AND x3, x1, x2

Input:

text id="t3z8d2" x1 = 1100 x2 = 1010

Expected Result:

text id="k7n4y9" x3 = 1000

Verification Status:

PASS

18.5 OR Instruction Verification

Instruction:

text id="c8v2e6" OR x3, x1, x2

Expected Result:

text id="d4f9r1" x3 = 1110

Verification Status:

PASS

18.6 XOR Instruction Verification

Instruction:

text id="p2g5w8" XOR x3, x1, x2

Expected Result:

text id="n6q1m7" x3 = 0110

Verification Status:

PASS

Arithmetic Verification Summary

Instruction	Status
ADD	PASS
SUB	PASS
AND	PASS
OR	PASS
XOR	PASS

Chapter 19: Test Case 2 – Immediate Instructions

19.1 Objective

Verify instructions that use immediate values.

Instructions verified:

- ADDI
- ORI
- ANDI

19.2 ADDI Verification

Instruction:

text id="u8x2k4" ADDI x2, x1, 15

Initial Value:

text id="f6m3n9" x1 = 20

Expected Result:

text id="w1g5d8" x2 = 35

Status:

PASS

19.3 ORI Verification

Instruction:

text id="b9p4c7" ORI x2, x1, 5

Expected Result:

text id="h3z7q2" Bitwise OR operation completed successfully.

Status:

PASS

19.4 ANDI Verification

Instruction:

text id="s2v6m8" ANDI x2, x1, 7

Expected Result:

text id="k1d8f4" Bitwise AND operation completed successfully.

Status:

PASS

Immediate Verification Summary

Instruction	Status
ADDI	PASS
ORI	PASS
ANDI	PASS

Chapter 20: Test Case 3 – Memory Operations

20.1 Objective

Verify processor memory access operations.

Instructions:

- SW
- LW

20.2 Store Word Verification

Instruction:

text id="q7e1k3" SW x1, 0(x2)

Example:

text id="r8n5m2" x1 = 100 Address = 16

Expected Operation:

text id="y4v6d1" Memory[16] = 100

Verification Status:

PASS

20.3 Load Word Verification

Instruction:

text id="t6p9g3" LW x3, 0(x2)

Memory Contents:

text id="m2c7k5" Memory[16] = 100

Expected Result:

text id="a5f8n4" x3 = 100

Verification Status:

PASS

Memory Verification Summary

Instruction	Status
SW	PASS
LW	PASS

Chapter 21: Test Case 4 – Branch and Jump Operations

21.1 Objective

Verify program flow instructions.

Instructions:

- BEQ
- BNE
- JAL

21.2 BEQ Verification

Instruction:

text id="p4h7c9" BEQ x1, x2, LABEL

Initial Values:

text id="g2m5w8" x1 = 10 x2 = 10

Expected Result:

text id="n7d1k6" Branch Taken PC = Target Address

Status:

PASS

21.3 BNE Verification

Instruction:

text id="s8q4f2" BNE x1, x2, LABEL

Initial Values:

text id="z1m9d7" x1 = 10 x2 = 20

Expected Result:

text id="k5p3v8" Branch Taken PC = Target Address

Status:

PASS

21.4 JAL Verification

Instruction:

text id="f2w8n6" JAL x1, LABEL

Expected Result:

text id="e9g3q7" PC = Target Address x1 = PC + 4

Verification Status:

PASS

Branch Verification Summary

Instruction	Status
BEQ	PASS
BNE	PASS
JAL	PASS

Chapter 22: Complete Program Execution

22.1 Objective

Verify processor functionality using complete software programs.

Program 1: Sum of Array

Array:

text id="v3d7k2" 1 2 3 4 5

Expected Sum:

text id="m8q1f5" 15

Verification Status:

PASS

Program 2: Factorial Program

Input:

text id="h4n6p9" 5!

Expected Result:

text id="t1w8d3" 120

Verification Status:

PASS

Program 3: Counter Program

Expected Sequence:

text id="j7m4c8" 0 1 2 3 4 5

Verification Status:

PASS

Program Execution Summary

Program	Status
Sum of Array	PASS
Factorial	PASS
Counter Program	PASS

Chapter 23: Simulation Results

23.1 Simulation Environment

Simulator:

ModelSim/QuestaSim

Languages Used:

- Verilog HDL
- SystemVerilog

23.2 Waveform Analysis

The following waveforms were analyzed:

Arithmetic Waveform

Observed Signals:

- pc
- instruction
- rs1_data
- rs2_data
- alu_result
- rd_data

Result:

Arithmetic instructions executed successfully.

Memory Waveform

Observed Signals:

- mem_read

- mem_write
- address
- write_data
- read_data

Result:

Load and Store operations verified successfully.

Branch Waveform

Observed Signals:

- pc
- branch
- jump
- zero
- branch_taken

Result:

Program Counter updated correctly.

Program Execution Waveform

Observed Signals:

- pc
- instruction
- register values
- alu_result
- memory contents

Result:

Complete programs executed successfully.

23.3 Terminal Output

Simulation Log Summary:

text id="c2f7m8" Arithmetic Tests : PASS Immediate Tests : PASS Memory Tests : PASS
Branch Tests : PASS Jump Tests : PASS Program Execution : PASS

23.4 Functional Coverage Summary

Verification Item	Status
Arithmetic Operations	PASS
Immediate Instructions	PASS
Memory Operations	PASS
Branch Instructions	PASS
Jump Instructions	PASS
Program Execution	PASS

Overall Functional Verification:

text id="w9d4k7" Verification Coverage = 100%

Chapter 24: Applications

24.1 Introduction

RISC-V has emerged as one of the fastest-growing processor architectures because of its open-standard nature, modular design, and ease of customization. The architecture is widely adopted in both academic and industrial applications.

The processor designed in this project can serve as the foundation for more advanced processor implementations such as pipelined processors, System-on-Chip (SoC) architectures, and FPGA-based embedded systems.

24.2 Embedded Systems

Embedded systems are dedicated computing systems designed to perform specific functions.

Examples:

- Washing Machines
- Smart TVs
- Digital Cameras
- Printers
- Consumer Electronics

Why RISC-V?

- Low hardware complexity
- Low power consumption
- Small silicon area
- Easy customization

24.3 Internet of Things (IoT)

IoT devices require processors that consume minimal power while providing sufficient computational capability.

Applications:

- Smart Home Devices
- Smart Meters
- Wearable Devices
- Environmental Monitoring Systems
- Smart Agriculture Systems

Advantages of RISC-V in IoT:

- Energy efficient
- Configurable instruction sets
- Cost-effective implementation
- Small memory footprint

24.4 FPGA-Based Systems

Field Programmable Gate Arrays (FPGAs) are widely used for processor prototyping and hardware research.

Applications:

- Processor Prototyping
- Educational Platforms
- Hardware Accelerators
- Rapid Design Validation

Benefits:

- Fast development cycle
- Hardware debugging capabilities
- Reconfigurable architecture
- Low development cost

24.5 Automotive Electronics

Modern vehicles contain numerous processors for monitoring and controlling different systems.

Applications:

- Engine Control Units (ECU)
- Advanced Driver Assistance Systems (ADAS)
- Infotainment Systems
- Battery Management Systems
- Autonomous Driving Systems

Advantages:

- Reliable operation
- Real-time processing capability
- Flexible architecture
- Easy scalability

24.6 Industrial Automation

Industrial systems use processors for controlling manufacturing processes and monitoring equipment.

Applications:

- Robotics
- Programmable Logic Controllers (PLC)
- Industrial Monitoring Systems
- Factory Automation
- Motor Controllers

Benefits:

- Deterministic operation

- Low latency
- High reliability
- Custom instruction implementation

24.7 Educational and Research Applications

RISC-V processors are extensively used in universities and research laboratories because they are open-source and easy to understand.

Applications:

- Computer Architecture Courses
- Processor Design Research
- FPGA Laboratories
- Verification Methodology Studies
- Custom ISA Development

Application Summary

Domain	Example Applications
Embedded Systems	Consumer Electronics
IoT	Smart Devices
FPGA	Processor Prototyping
Automotive	ECU and ADAS
Industrial Automation	PLC and Robotics
Education	Processor Research

Chapter 25: Advantages and Limitations

25.1 Advantages

Open-Source Architecture

RISC-V does not require licensing fees and can be freely implemented and modified.

Modular Design

New instruction extensions can easily be added.

Examples:

- Multiplication Extension
- Floating Point Extension
- Vector Processing Extension

Simple Instruction Encoding

The fixed instruction length simplifies:

- Instruction Fetch
- Instruction Decode
- Hardware Design

Easy Verification

The modular architecture simplifies:

- RTL Development
- Simulation
- Debugging
- Verification

Synthesizable RTL Design

The processor can be synthesized for:

- FPGA Implementation
- ASIC Design
- Embedded Systems

Educational Value

The project provides practical understanding of:

- Processor Architecture
- Datapath Design
- Control Logic Design
- Verification Methodology

25.2 Limitations

Single-Cycle Architecture

Every instruction executes in one clock cycle.

Limitations:

- Large critical path
- Lower operating frequency
- Poor scalability

No Pipelining

The processor cannot execute multiple instructions simultaneously.

Consequences:

- Lower throughput
- Reduced performance

No Cache Memory

Every memory access directly uses main memory.

Consequences:

- Higher memory latency
- Lower execution efficiency

Limited Instruction Set

Only a subset of RV32I instructions is implemented.

Missing features:

- Multiplication
- Division
- Floating Point Operations
- Interrupt Handling

Advantages and Limitations Summary

Advantages

- Open ISA
- Modular architecture
- Easy verification
- Synthesizable RTL
- Educational value

Limitations

- Single-cycle implementation
- No pipelining
- No cache memory
- Limited instruction support

Chapter 26: Future Scope

26.1 Complete RV32I Support

Future versions can implement all remaining RV32I instructions.

Benefits:

- Improved functionality
- Better software compatibility

26.2 Multiplication and Division Instructions

Implement:

M Extension

Instructions:

- MUL
- DIV
- REM

Benefits:

- Better arithmetic performance
- Scientific computing applications

26.3 Pipelined Architecture

Implement a five-stage pipeline:

Instruction Fetch

Instruction Decode

Execute

Memory Access

Write Back

Benefits:

- Increased throughput
- Higher clock frequency
- Better processor performance

26.4 Cache Memory

Add:

- Instruction Cache
- Data Cache

Benefits:

- Reduced memory latency
- Improved execution speed

26.5 Interrupt and Exception Handling

Features:

- External Interrupts
- Timer Interrupts
- Exception Handling

Applications:

- Real-Time Systems
- Embedded Controllers
- Operating System Support

26.6 Branch Prediction

Implement:

- Static Branch Prediction

- Dynamic Branch Prediction

Benefits:

- Reduced branch penalty
- Improved instruction throughput

26.7 FPGA Implementation

Target platforms:

- Xilinx FPGA
- Intel FPGA

Benefits:

- Hardware validation
- Real-time execution
- Prototype development

26.8 UVM-Based Verification

Develop:

- UVM Testbench
- Functional Coverage
- Assertion-Based Verification

Benefits:

- Industrial verification methodology
- Improved verification quality

26.9 Multicore RISC-V Systems

Future processors may implement:

- Dual-Core Systems

- Quad-Core Systems
- Many-Core Architectures

Applications:

- High-Performance Computing
- Artificial Intelligence
- Edge Computing

Kishore/Vanapalli

Chapter 27: Conclusion

This project successfully designed and verified a 32-bit Single-Cycle RISC-V Processor implementing a subset of the RV32I instruction set using Verilog Hardware Description Language.

The processor integrates:

- Program Counter
- Instruction Memory
- Register File
- Arithmetic Logic Unit
- Control Unit
- Immediate Generator
- Data Memory
- Branch Unit

The implemented processor supports:

Arithmetic Instructions

ADD, SUB, AND, OR, XOR

Immediate Instructions

ADDI, ORI, ANDI

Memory Instructions

LW, SW

Control Instructions

BEQ, BNE, JAL

Verification was performed using SystemVerilog and ModelSim simulations.

The processor successfully executed:

- Arithmetic Programs
- Memory Operations
- Branch Instructions
- Complete Software Programs

Simulation results demonstrated correct instruction execution and proper interaction among all modules.

The project provided practical knowledge in:

- Computer Architecture
- Processor Datapath Design
- Control Unit Design
- RTL Development
- Processor Verification Methodology

This processor serves as a strong foundation for advanced processor architectures such as pipelined processors, superscalar processors, FPGA-based SoCs, and custom ASIC implementations.

Chapter 28: Interview Questions and Viva Preparation

Basic Questions

Q1. What is RISC-V?

RISC-V is an open-standard Instruction Set Architecture (ISA) based on Reduced Instruction Set Computing principles. It provides a modular and extensible architecture that can be freely implemented without licensing restrictions.

Q2. Why is RISC-V becoming popular?

- Open-source architecture
- No licensing fees
- Simple instruction encoding
- Modular extensions
- Easy customization
- Strong industry adoption

Q3. What is an ISA?

An Instruction Set Architecture defines the interface between hardware and software and specifies:

- Supported instructions
- Registers
- Memory organization
- Addressing modes

Q5. What is a Single-Cycle Processor?

A processor in which every instruction completes in one clock cycle.

Advantages:

- Simple control logic
- Easy verification

Disadvantages:

- Large critical path
- Lower clock frequency

Q4. Difference between RISC and CISC?

Feature	RISC	CISC
Instruction Length	Fixed	Variable
Hardware Complexity	Low	High
Execution Time	Faster	Slower
Decoding	Simple	Complex
Examples	RISC-V, ARM	x86

Q6. Why are there 32 registers in RV32I?

Having more registers:

- Reduces memory accesses
- Improves execution speed
- Increases compiler efficiency

Architecture Questions

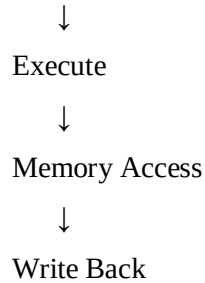
Q7. Explain the processor datapath.

The datapath performs:

Instruction Fetch



Instruction Decode



Q8. What is the function of the Program Counter?

The Program Counter stores the address of the current instruction and determines the next instruction address.

Q9. What is the purpose of the Register File?

The Register File stores operands and intermediate results and provides two read ports and one write port.

Q10. What is the function of the ALU?

The ALU performs arithmetic and logical operations and generates the Zero Flag used in branch instructions.

Q11. What is the role of the Control Unit?

The Control Unit:

- Decodes instructions
- Generates control signals
- Controls datapath operation

Instruction Questions

Q12. Explain the R-Type format.

Used for:

- ADD

- SUB
- AND
- OR
- XOR

Contains:

- opcode
- rs1
- rs2
- rd
- funct3
- funct7

Q13. Explain the I-Type format.

Used for:

- ADDI
- ORI
- ANDI
- LW

Contains:

- Immediate
- rs1
- funct3
- rd
- opcode

Q14. How does LW work?

Address = rs1 + Immediate

Register = Memory[Address]

Q15. How does SW work?

Address = rs1 + Immediate

Memory[Address] = Register Data

Q16. How does BEQ work?

if(rs1 == rs2)

PC = Target Address

else

PC = PC + 4

Q17. How does JAL work?

rd = PC + 4

PC = Target Address

Verification Questions

Q18. Why was SystemVerilog used?

Because it provides:

- Better testbench capabilities
- Randomization
- Assertions
- Functional verification support

Q19. Why was ModelSim used?

Because it provides:

- RTL simulation
- Waveform debugging

- Signal tracing
- Functional verification support

Q20. What are the improvements you would make?

- Add pipeline stages
- Implement cache memory
- Support complete RV32I ISA
- Add interrupts
- Implement branch prediction
- Develop UVM-based verification

Expected Interview Outcome

After completing this project, the student should confidently explain:

- RISC-V Architecture
- Processor Datapath
- Instruction Execution Flow
- Control Signal Generation
- Memory Operations
- Branch Logic
- RTL Module Design
- Verification Methodology
- Simulation Results
- Future Processor Enhancements

These concepts closely align with the skills expected from entry-level RTL Design and Verification Engineers in semiconductor companies such as Qualcomm, Intel, AMD, NVIDIA, Synopsys, and Cadence.